

Domain Transform Investigation – Basis Images, Energy Compaction, and Filtering

Analytical Questions

Objective

Understanding the visual and practical aspects of 2D Unitary Transforms, basis images, energy compaction, and how manipulating transform coefficients affects image quality using standard Python tools.

Part 1: Construction and Visualization of Basis Images

Theory

Each frequency coefficient $F(k_1, k_2)$ corresponds to a "basis image" called A_{k_1, k_2} . An interesting property of unitary transforms is that if you create a matrix in the frequency domain where all entries are zero except for the entry (k_1, k_2) which is set to 1.0 (an impulse signal at the desired frequency), then taking the inverse transform (IDCT) of that matrix will exactly produce the "basis image" corresponding to that frequency.

Steps

1. Using the `idct2` function introduced in the technical guide, write a program that generates all 64 basis images A_{k_1, k_2} (with dimensions 8×8) for $N = 8$.

Hint: In a double loop, create an 8×8 zero matrix, set the desired frequency coefficient to 1, and take the inverse transform.

2. **Visualization:** Create an 8×8 grid of subplots and display the corresponding basis image in each cell.

Important note: Since basis images contain both positive and negative values, use a **bipolar colormap** (e.g., `cmap='RdBu'` or `cmap='coolwarm'`) and set the display range symmetrically around zero (parameters `vmin=-0.5` and `vmax=0.5`).

Analytical Question

Look at the grid of basis images. Which ones correspond to low frequencies (smooth variations) and which ones correspond to high frequencies (rapid oscillations)? How does this relate to the row and column indices (k_1 and k_2) in the frequency matrix?

Part 2: Implementation of 2D Transform and Separability Investigation

Theory

Symmetric and separable transforms can be applied sequentially on rows and columns, which is exactly what the `dct2` and `idct2` functions do optimally.

Steps

1. Load the grayscale `camera` image and resize it to 256×256 .
2. Using the `dct2` function, compute the forward transform of the image to obtain the coefficient matrix F .
3. Using the `idct2` function, apply the inverse transform to the matrix F to obtain the reconstructed image $f_{\text{reconstructed}}$.
4. **Verification:** Calculate the Mean Squared Error (MSE) between the original image f and the reconstructed image $f_{\text{reconstructed}}$. This value should be at the level of floating-point error (close to absolute zero). What conclusion do you draw from this?

Analytical Question

Given that we use the `norm='ortho'` argument in the ready-made functions, explain why this parameter is mathematically critical and how it helps us preserve the properties of a unitary transform (such as energy preservation and perfect reconstruction).

Part 3: Energy Compaction and Image Reconstruction with Limited Coefficients

Theory

The DCT transform concentrates image energy in a small number of coefficients (primarily low frequencies).

Tasks

1. Display the logarithmic magnitude spectrum of the image coefficients, $\log(1 + |F|)$, as an image.
Note: In DCT, the zero-frequency (DC) component is located at the top-left corner at coordinates $(0,0)$, and there is no need to shift frequencies to the center.
2. **Compression Experiment:** Keep only a certain percentage of the larger coefficients (based on the absolute value of matrix F) and set the rest to zero. Perform this for percentages of 1%, 5%, 10%, 25%, and 50%.

3. For each case, obtain the reconstructed image using the `idct2` function (make sure to clip the output values between 0 and 1 using `np.clip`).
4. **Visualization:** Display the original image alongside the 5 reconstructed images in a single row. Write the percentage of retained coefficients and the PSNR evaluation metric on the title of each image.
5. Plot a line graph of the PSNR metric against the "percentage of retained coefficients".

Analytical Question 1

In the case of keeping only 5% of the coefficients, how is the visual quality of the image? What does this phenomenon teach us about the efficiency of DCT in compression systems (like JPEG)?

Analytical Question 2

In the frequency domain visualization step, instead of directly plotting the magnitude of coefficients ($|F|$), we used the logarithmic magnitude spectrum $\log(1 + |F|)$. Temporarily modify your code and try to directly plot the $|F|$ matrix itself. How does the visual output appear in this non-logarithmic case? Considering the very large magnitude of the zero-frequency (DC) component compared to other coefficients, explain the mathematical reason for needing this nonlinear (logarithmic) function for visual representation of the frequency spectrum.

Part 4: Filtering in the Transform Domain

Theory

Since the transform domain separates frequencies based on their geometric location, we can create frequency filters by manipulating the coefficient matrix.

Tasks

1. Create a **low-pass mask** M_{LP} . Given that the DC frequency is at coordinates (0,0) (top-left corner), keep the coefficients whose geometric distance from the origin is less than or equal to radius r ($\sqrt{k_1^2 + k_2^2} \leq r$) and set the rest to zero. Perform this for two radii: $r = N/8$ and $r = N/4$ (where N is the image dimension).
2. Create a **high-pass mask** M_{HP} that zeros out the coefficients within radius $r = N/8$ and leaves the rest untouched.
3. Apply these masks to the matrix F (element-wise multiplication) and reconstruct the images using `idct2`.

Analytical Question 1

In the output of low-pass filters, what visual changes (blurring, detail removal) do you observe? Why does the quality change as the radius increases?

Analytical Question 2

What features of the image (edges, textures) does the high-pass filter output reveal?

Analytical Question 3

Referring to the basis images you generated in Part 1, analyze the reason for this visual behavior.

Submission Requirements

Upload your answer as a compressed `.zip` file named `IP_StudentName_StudentId` to the system. This compressed file must exactly contain:

1. Python file or Jupyter notebook
2. A PDF file containing answers to the analytical questions, screenshots of visual results, and comparative subplots

Technical Guide: Using DCT Functions

In this exercise, you don't need to manually write the mathematical transform matrices. You will use the ready-made DCT functions from the `scipy` library.

Since images are 2D, you need to apply the 1D transform once on rows and once on columns. Define the following 2D functions at the beginning of your code:

```
from scipy.fftpack import dct, idct

def dct2(image):
    # Apply transform on columns (axis=0) and then on rows (axis=1)
    # The norm='ortho' parameter ensures the transform is unitary and energy-preserving
    return dct(dct(image, axis=0, norm='ortho'), axis=1, norm='ortho')

def idct2(coefficients):
    # Apply inverse transform on columns and then on rows
    return idct(idct(coefficients, axis=0, norm='ortho'), axis=1, norm='ortho')
```

Key Implementation Notes

- Use `skimage.data.camera()` for loading the image
- Use `skimage.transform.resize()` for dimension adjustment
- Use `np.clip()` to ensure pixel values are in $[0,1]$ range
- Use `sklearn.metrics.mean_squared_error` for MSE calculation
- Use `matplotlib.pyplot` for all visualizations